Python Methods for Least Squares Regression

- **Key Python Libraries**:
  - Several libraries in Python can be used for least squares regression: Numpy and Scipy
    - **Numpy**: Provides functions such as numpy.linalg.lstsq() and numpy.linalg.pinv() for solving the least squares problem.
    - **Scipy**: Contains optimize.curve_fit() for more advanced fitting, including non-linear regression.
- **Example Workflow (Using Numpy)**:
  - **Generate data, Form the matrix A, Solve using least square**

```python
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))
```

$$\hat{y} = \alpha_1 x + \alpha_2$$

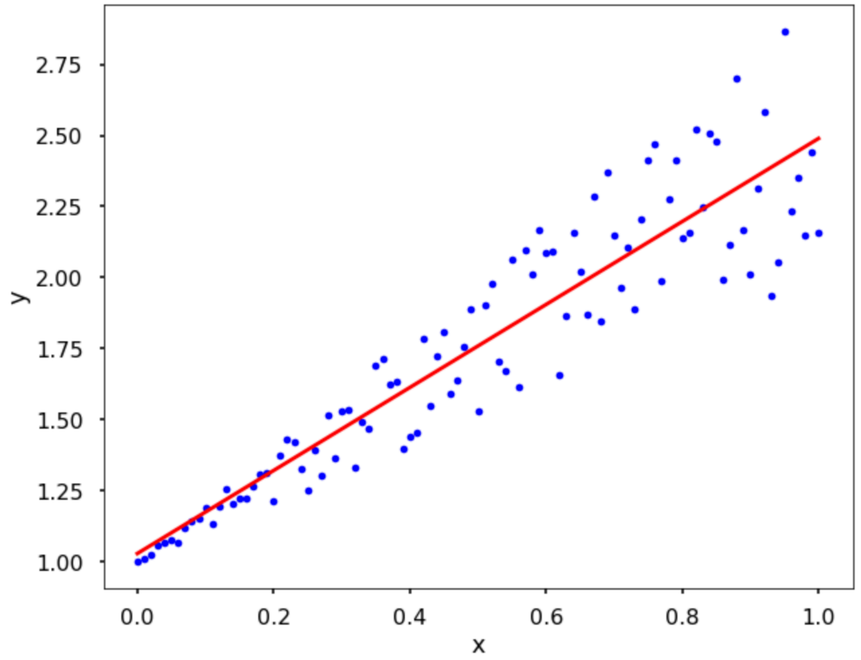
```python
# assemble matrix A
A = np.vstack([x, np.ones(len(x))]).T

# turn y into a column vector
y = y[:, np.newaxis]
```

$$\beta = (A^T A)^{-1} A^T Y$$

```python
# Direct least square regression
alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T,A)),A.T)),y)
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```



  - **Plot the results**:

```python
plt.plot(x, y, 'b.', x, alpha[0]*x + alpha[1], 'r')
```

Use numpy.linalg.lstsq

```
alpha = np.linalg.lstsq(A, y, rcond=None)[0]
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```

Use optimize.curve_fit from scipy

using this function, we don't need to turn y into a column vector.

```
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))
```

```
def func(x, a, b):
    y = a*x + b
    return y

alpha = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(alpha)
```

```
[1.44331612 1.0396133 ]
```

Least Squares Regression for Nonlinear Functions

- **Definition**: Least squares regression estimates parameters by minimizing the sum of squared differences between observed and predicted data points.

- **Nonlinear Functions**: Some functions can't be modeled using traditional linear combinations of basis functions but can still benefit from least squares regression.

- **Methods for Nonlinear Functions**:
  1. **Logarithmic Transformations**: Convert nonlinear functions (e.g., exponential and power functions) to a linear form by applying logarithms.
  2. **Direct Nonlinear Fitting**: Using methods like curve_fit from scipy to fit a nonlinear model without transforming the function.

Logarithmic Tricks for Nonlinear Models

1. **Exponential Functions**:

• Given an exponential function, we can linearize it using logarithms:

$$y = \alpha e^{\beta x}$$

$$\log(y) = \log(\alpha) + \beta x$$

•Perform least squares on the linearized form, then recover parameters.

2. **Power Functions**:

• Similar approach for power functions

$$y = bx^m$$

$$\log(y) = m\log(x) + \log(b)$$

•After transforming, we apply least squares regression to solve for the parameters m and b.

Other Techniques for Nonlinear Regression

1. **Polynomial Regression**:

- Use least squares for polynomials of varying orders:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

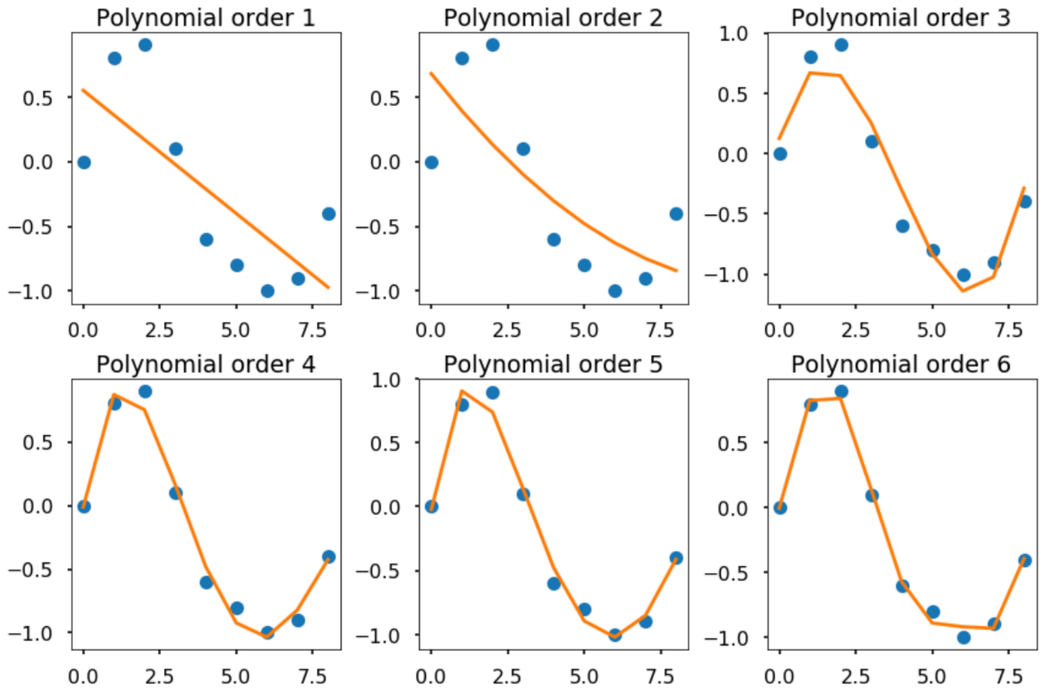
- Higher-order polynomials can be more flexible but risk overfitting.

```
x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1, -0.9, -0.4])

plt.figure(figsize = (12, 8))
for i in range(1, 7):

    # get the polynomial coefficients
    y_est = np.polyfit(x_d, y_d, i)
    plt.subplot(2,3,i)
    plt.plot(x_d, y_d, 'o')
    # evaluate the values for a polynomial
    plt.plot(x_d, np.polyval(y_est, x_d))
    plt.title(f'Polynomial order {i}')

plt.tight_layout()
plt.show()
```



Other Techniques for Nonlinear Regression

2. **Direct Nonlinear Fitting with** curve_fit:

- The curve_fit function from scipy directly estimates parameters for nonlinear functions without requiring transformations.

- Example: fitting an exponential function. $y = \alpha e^{\beta x}$

```
# let's define the function form
def func(x, a, b):
    y = a*np.exp(b*x)
    return y

alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f'alpha={alpha}, beta={beta}')
```

```
alpha=0.12420824748558632, beta=0.28005793982974525
```

```
# Let's have a look of the data
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha*np.exp(beta*x), 'r')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

